SET 5

PART A — Short questions and answers

Q1. Explain why PURCHASE_ITEM is treated as a weak entity that depends on PURCHASE.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Describe how a CHECK constraint can prevent a negative Stock_Qty.

Answer: A CHECK constraint accepts only values that satisfy a specified condition.

Q3. Describe a transitive dependency using Product_ID → Supplier_ID → Supplier_City.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain a subquery using an example that lists products priced above the average price.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Compare the WHERE clause and the HAVING clause.

Answer: HAVING filters groups after GROUP BY, often using aggregate conditions such as COUNT(*) > 3.

Q6. Explain the purpose of the SELECT ... INTO statement in a PL/SQL block.

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Explain how embedded documents reduce the need for joins.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Compare the insertOne() and insertMany() methods.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Explain what the following MongoDB query returns: db.products.find({StockQty: {$lt: 20}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Describe the purpose of WHERE in: MATCH (p:Product) WHERE p.price > 500 RETURN p.name;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A wholesale warehouse maintains a Warehouse Inventory Management System to store product details,

1(a) ER design:

• **PRODUCT**(ProductID, ProductName, Category, UnitPrice, StockQty); primary key: **ProductID**.

• **SUPPLIER**(SupplierID, SupplierName, City, Phone); primary key: **SupplierID**.

• **PURCHASE**(PurchaseID, ProductID, SupplierID, PurchaseDate, Quantity); primary key: **PurchaseID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE PRODUCT (
  ProductID NUMBER PRIMARY KEY,
  ProductName VARCHAR2(100),
  Category VARCHAR2(100),
  UnitPrice VARCHAR2(100),
  StockQty VARCHAR2(100)
);
CREATE TABLE SUPPLIER (
  SupplierID NUMBER PRIMARY KEY,
```

```
  SupplierName VARCHAR2(100),
  City VARCHAR2(100),
  Phone VARCHAR2(100)
);
CREATE TABLE PURCHASE (
  PurchaseID NUMBER PRIMARY KEY,
  ProductID NUMBER,
  SupplierID NUMBER,
  PurchaseDate DATE,
  Quantity NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM PURCHASE x
JOIN PRODUCT a ON x.ProductID = a.ProductID
JOIN SUPPLIER b ON x.SupplierID = b.SupplierID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```